

Kubernetes Networking Masterclass

A Comprehensive, Concept-by-Concept Guide

Generated for Deep-Dive Learning

1. The Core IPs: Node, Pod, and Cluster IPs

What is the difference between Pod IP, Service IP and Node IP?

To understand Kubernetes networking, you must differentiate between the three primary IP spaces:

- **Node IP:** This is the physical or virtual IP address of the host machine (e.g., an AWS EC2 instance). It sits on your actual VPC subnet. It is completely permanent until the machine is destroyed.
- **Pod IP:** This is the IP assigned directly to a container (Pod). It is part of a virtual, software-defined network overlaid on top of your physical network. It is strictly internal to the cluster.
- **Service IP (Cluster IP):** This is a completely virtual IP that doesn't belong to any specific network interface. It is a stable load-balancing frontend mapped in `iptables` that routes traffic to a rotating cast of Pod IPs.

Why does a Pod IP change when a pod is recreated?

In Kubernetes, Pods are treated as "cattle, not pets." They are completely ephemeral. When a deployment is updated, a node crashes, or scaling occurs, Kubernetes terminates the existing Pods and creates entirely new ones.

When a new Pod spins up, the networking plugin (CNI) looks at its internal IP Address Management (IPAM) pool and hands out the next available IP address. It does not reuse the old IP immediately. Because of this high churn rate, relying on Pod IPs for application communication is an anti-pattern. You must always use Services.

Network Engineer Perspective:

Imagine an office where employees (Pods) connect their laptops to WiFi via DHCP. Every time an employee closes their laptop and opens it again, they might get a new IP address. You would never hardcode an employee's IP address into a

firewall rule; instead, you would route to a stable proxy (Service IP) that knows where the employee currently is.

2. Container Network Interface (CNI)

What is CNI (Container Network Interface)?

Kubernetes itself does not natively handle networking. It relies on third-party plugins that adhere to the **Container Network Interface (CNI)** specification. The CNI acts as the translator between Kubernetes and the underlying Linux network stack.

When the `kubelet` on a node creates a Pod, it calls the installed CNI plugin (e.g., Calico, Flannel, AWS VPC CNI, Cilium) and says, "I have a new container, please attach it to the network."

The CNI is responsible for two main things:

- **IPAM (IP Address Management):** Assigning a unique IP address to the Pod from a predefined CIDR block.
- **Routing & Interfaces:** Creating a virtual ethernet (veth) pair. One end goes inside the container's network namespace, and the other end connects to a virtual bridge (like `cbr0`) on the host Node.

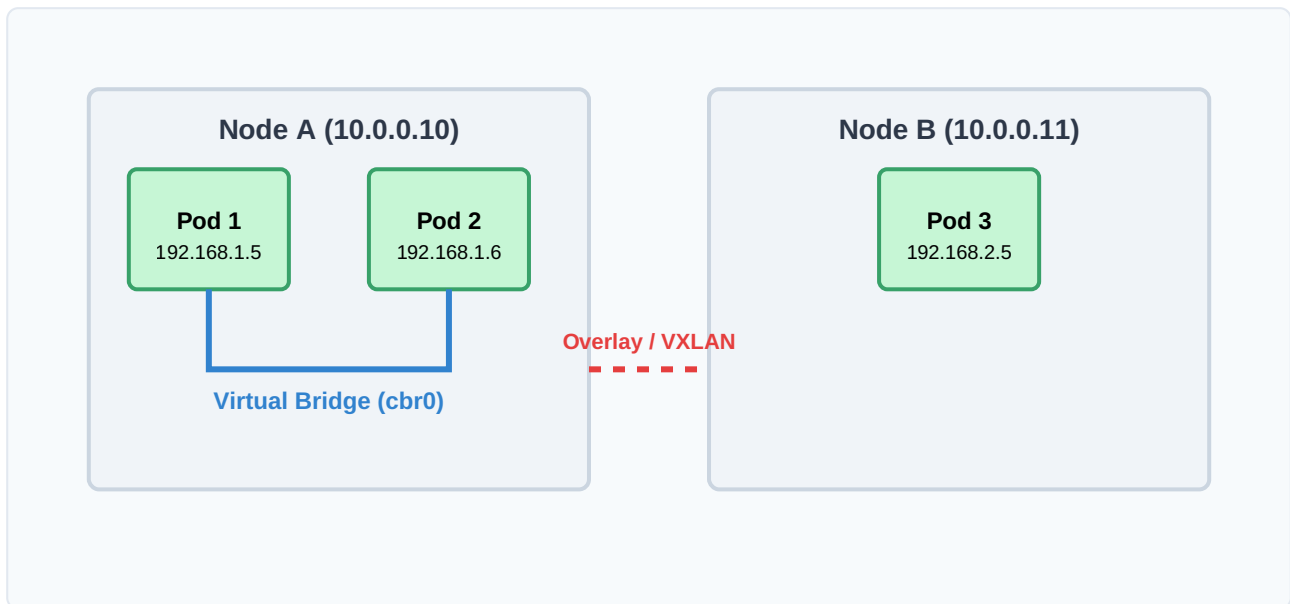
Real-World Examples:

Flannel: A very simple CNI that uses VXLAN to create an overlay network.

Calico: A robust CNI that uses BGP (Border Gateway Protocol) for direct routing and supports advanced Network Policies (firewalling).

AWS VPC CNI: Gives Pods real IP addresses directly from the AWS VPC subnet, completely skipping the overlay network.

3. Pod-to-Pod Communication



How does networking work between two pods? (Same node vs Different nodes)

Kubernetes imposes a strict networking requirement: **All Pods must be able to communicate with all other Pods without NAT (Network Address Translation).**

Scenario 1: Pods on the Same Node

If Pod 1 (192.168.1.5) wants to talk to Pod 2 (192.168.1.6) on the same Node, the packet never leaves the physical server. The CNI has connected both Pods to a virtual ethernet bridge (like `cbr0`). The packet leaves Pod 1's virtual interface, hits the bridge, and is directly switched to Pod 2's virtual interface. It is purely Layer 2 switching.

Scenario 2: Pods on Different Nodes

If Pod 1 wants to talk to Pod 3 (192.168.2.5) on Node B, the process is more complex:

- The packet leaves Pod 1, hits the virtual bridge on Node A, and realizes the destination IP is not on the local bridge.
- The packet is routed to the Node A's default network interface.
- Here, the CNI takes over. Because Node A's physical network does not know how to route `192.168.2.x` traffic, the CNI encapsulates the packet inside a new packet (an overlay tunnel, often VXLAN or IP-in-IP). The new packet's destination is Node B's physical IP (10.0.0.11).
- The packet travels across the physical network to Node B.
- Node B receives the packet, the CNI strips away the outer wrapper (decapsulation), reveals the original packet meant for Pod 3, and delivers it to Pod 3 via the local bridge.

4. Services and Selectors

How does a Service find the correct pods? What is a selector?

Because Pod IPs constantly change, Kubernetes uses an abstraction called a **Service** to provide a stable network endpoint. A Service is deeply tied to a **Selector**.

A Selector is a simple key-value matching system defined in the Service YAML configuration. For example, you might define a Service with the selector `app: payment-processor`.

The Discovery Process:

- When you create the Service, a Kubernetes component called the Endpoint Controller constantly monitors the cluster for any Pods that have labels exactly matching `app: payment-processor`.
- When it finds these Pods, it takes their dynamic IP addresses and adds them to an object called an **EndpointSlice** (or Endpoints).
- If a Pod dies, the controller immediately removes its IP from the EndpointSlice. If a new Pod spins up with that label, its new IP is added.
- Therefore, the Service always knows exactly which Pod IPs to route traffic to, in real-time, completely dynamically.

5. kube-proxy & Traffic Routing

What is kube-proxy? What is iptables / IPVS?

kube-proxy is the workhorse of Kubernetes networking. It is a daemon (background process) that runs on *every single Node* in the cluster.

While the API server creates the Service IP and tracks the Pod endpoints, **kube-proxy** is the component that actually makes the routing happen on the Linux operating system level.

kube-proxy continuously talks to the Kubernetes API. When it sees that a new Service has been created (or Pod endpoints have changed), it translates that information into low-level network rules on its host Node using either **iptables** or **IPVS**.

- **iptables mode:** kube-proxy writes a series of NAT (Network Address Translation) rules into the Linux kernel. When a packet tries to go to a Service IP (e.g., 10.96.0.10), the iptables rules intercept it and say, "Change the destination IP to one of these three Pod IPs using a random probability."
- **IPVS mode:** Designed for massive clusters. It uses the Linux kernel's IP Virtual Server module, which acts as a true Layer 4 load balancer. It is much faster than iptables and supports better load balancing algorithms (like round-robin or least-connections).

What happens if kube-proxy is down?

If the **kube-proxy** process crashes on a Node, the immediate impact depends on the state of the kernel:

Because **kube-proxy** writes rules into the OS kernel (iptables), those rules stay in the kernel even if kube-proxy dies. Existing Services will temporarily continue to work.

However: If any Pods scale up, scale down, or crash, the iptables rules will not be updated. Traffic will start being routed to dead Pod IPs (causing connection timeouts), and new Pods will not receive any traffic. Any brand new Services created will be completely unreachable from that Node.

6. Service Discovery via DNS

What is CoreDNS?

While Service IPs are stable, humans and applications prefer names over numbers. **CoreDNS** is the official DNS server deployed by default inside every Kubernetes cluster.

Every time you create a Service, CoreDNS automatically generates a DNS record for it. The standard naming convention is:

```
<service-name>.<namespace>.svc.cluster.local
```

When Pods are created, the `kubelet` automatically injects the CoreDNS Service IP into the Pod's `/etc/resolv.conf` file. This means any network call made by your application automatically queries CoreDNS first.

How does Service-to-Service communication work?

Let's trace a full request from a Frontend Pod to a Backend Pod:

- The Frontend code makes an HTTP request to `http://backend-service`.
- The Pod's OS queries CoreDNS: "What is the IP for backend-service?"
- CoreDNS responds with the Backend's Virtual Service IP (e.g., 10.100.20.30).
- The Frontend Pod sends a packet destined for 10.100.20.30.
- As the packet hits the Node's networking stack, the `iptables` rules (managed by kube-proxy) intercept the packet.
- iptables performs Destination NAT (DNAT), rewriting the destination IP from 10.100.20.30 to the actual IP of a living Backend Pod (e.g., 192.168.3.50).

- The CNI routes the packet to that specific Pod.

What happens if CoreDNS is down?

If the CoreDNS Pods crash, internal Service Discovery halts entirely. Your applications will suddenly start throwing ``NXDOMAIN`` (Non-Existent Domain) or ``Unknown Host`` errors when they try to contact other microservices via their names.

Interestingly, the underlying network is still perfectly fine. If an application was hardcoded to use a Service IP or Pod IP (which is bad practice), that traffic would still flow. Only the name-to-IP translation breaks.

7. External Access (Internet to Pod)

What is NodePort vs LoadBalancer vs Ingress?

These are the three primary ways to expose your internal Pods to the outside world.

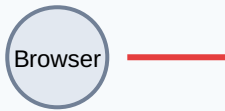
- **NodePort:** The most primitive method. Kubernetes opens a specific port (ranging from 30000 to 32767) on *every single Node's physical IP address*. If you hit any Node IP on that specific port, iptables forwards the traffic to your internal Service. It is rarely used in production due to security and port-management complexities.
- **LoadBalancer:** This builds on top of NodePort. When you create a Service of type LoadBalancer, Kubernetes reaches out to your Cloud Provider's API (like AWS or GCP) and automatically provisions an external, physical Load Balancer (like an AWS Network Load Balancer). The cloud LB is configured to point at the NodePorts on your cluster nodes.
- **Ingress:** A LoadBalancer operates at Layer 4 (TCP/UDP). **Ingress** operates at Layer 7 (HTTP/HTTPS). Instead of buying a new Cloud Load Balancer for every single microservice (which gets expensive), you buy ONE LoadBalancer that points to an **Ingress Controller**.

What is an Ingress Controller?

An Ingress resource is just a YAML file containing routing rules (e.g., "If URL path is /api, go to backend-service. If URL path is /store, go to store-service").

The **Ingress Controller** is the actual application (usually NGINX, HAProxy, or Traefik) running inside a Pod in your cluster that reads those YAML rules and acts as an intelligent reverse proxy, routing incoming HTTP traffic based on hostnames and URL paths.

8. End-to-End Traffic Flows



What happens when you access `https://example.com` hosted on Kubernetes? (How a request reaches a pod from the internet)

This is a classic interview question. Here is the step-by-step flow:

DNS: The user's browser queries public DNS for `example.com`. It returns the public IP address of your Cloud Provider's Load Balancer (e.g., AWS ALB).

Cloud Load Balancer: The traffic hits the AWS Load Balancer. The LB is configured to forward this traffic to the NodePorts of your Kubernetes worker nodes.

Node to Ingress Controller: The traffic arrives at a Node. The Node's iptables rules forward the packet to the Pod running your **Ingress Controller** (like NGINX).

Layer 7 Routing: NGINX decrypts the HTTPS traffic (SSL Termination). It reads the HTTP Host header (`example.com`) and looks up its Ingress routing rules. It sees that `example.com` should be sent to `frontend-service`.

Service to Pod: NGINX forwards the traffic to the Cluster IP of `frontend-service`. Kube-proxy's iptables intercept this, translate the

Service IP to an actual **Pod IP**, and the packet is delivered to your application container!

Pod-to-Internet communication (Egress)

What if your Pod needs to reach out to the internet to download a software update? Pod IPs are strictly private and unroutable on the internet. When the packet leaves the Pod and hits the Node's physical network interface, the Node performs **SNAT (Source Network Address Translation)**. It replaces the Pod's private IP with the Node's physical IP. When the internet responds, the Node translates the packet back and routes it to the Pod.

9. Security: Network Policies

What is NetworkPolicy?

By default, Kubernetes clusters are "flat." This means any Pod in any Namespace can communicate with any other Pod in the cluster. From a security standpoint, this is highly dangerous.

A **NetworkPolicy** is a Kubernetes resource that defines how groups of Pods are allowed to communicate with each other and other network endpoints. It is essentially a software-defined firewall (Layer 3 / Layer 4) applied directly to Pods via the CNI plugin (Note: Not all CNIs support NetworkPolicies; Calico does, Flannel natively does not).

How do you restrict one namespace from communicating with another?

To restrict traffic, you implement a "Default Deny" posture and then whitelist specific traffic.

Step 1: Default Deny. You apply a NetworkPolicy in your target namespace that selects all Pods (using an empty `podSelector: {}`) and specifies no ingress rules. This immediately blocks all incoming traffic to all Pods in that namespace.

Step 2: Whitelist specific Namespaces. You create another NetworkPolicy that explicitly allows traffic. You define a `namespaceSelector` rule in the ingress section. For example, if you want the `database` namespace to only accept traffic from the `backend` namespace, you apply a policy to the database pods that allows ingress only if the source namespace has the label `name: backend`.

Network Engineer Perspective:

NetworkPolicies are equivalent to Access Control Lists (ACLs) applied at the switch port level. Instead of writing ACLs based on subnets or rigid IP addresses, you write them based on metadata labels.